

Name: Winnie Delinois

Date: 16 September 2026

Tools: Wireshark, Firefox, Python

Project: Network Troubleshooting Analysis Lab

Project Goal:

The purpose of this project was to use Wireshark to analyze a successful and unsuccessful network connection, in a controlled environment using python. The project focused on identifying the TCP three-way handshake, analyzing TCP flags and port numbers, observing HTTP application data, and using packet-level evidence to troubleshoot why a connection failed

Along with this, the main networking concepts shown in this project were TCP connections, TCP flags, TCP ports, HTTP communication, TCP reset (RST) packets, and Wireshark display filters.

Project Summary:

In this project I used a controlled environment to test a successful and failed connection. By using a local Python HTTP server and Wireshark, I first started the Python web server on port 8080 and used Firefox to access the server through the loopback address 127.0.0.1. Wireshark was then used to capture and analyze the packets generated during the successful connection. After analyzing the successful connection, I stopped the Python web server from the terminal, and tried to access the same IP address again. Since the server was no longer running on port 8080 anymore, the second connection attempt failed. I lastly used Wireshark to then compare the successful to the failed connections to identify their differences at a packet level

2. Lab Environment

Tools Used:

- Wireshark
- Firefox
- Python
- TCP/IP
- HTTP
- Ubuntu virtual machine

Lab setup:

The python web server was running in my Ubuntu virtual machine and was set to listen on port 8080. I used Firefox as the client to connect to the local server and through the loopback address 127.0.0.1.

The specific URL used to access the server:

`http://127.0.0.1:8080`

Item:

Information:

Client IP	127.0.0.1
Server IP	127.0.0.1
Server Port	8080

Protocol	TCP
Browser	Firefox
Web Server	Python HTTP server

3. Investigation 1 — Successful TCP Connection

Objective:

The goal of this investigation was to determine whether a connection could be successfully established between Firefox and the Python web server that was running on port 8080. I wanted to observe the TCP three-way handshake and determine whether or not HTTP application data was exchanged after the connection was made

What was the server/destination IP address?

127.0.0.1

What was the client's source port?

55084

What was the server's destination port?

8080

3.2 TCP Three-Way Handshake

Did you find a SYN packet?

Yes

Did you find a SYN/ACK packet?

Yes

Packet	Source IP	Destination IP	Source Port	Destination Port	TCP Flags
SYN	127.0.0.1	127.0.0.1	55084	8080	SYN
SYN/ACK	127.0.0.1	127.0.0.1	8080	55084	SYN/ACK
ACK	127.0.0.1	127.0.0.1	55084	8080	ACK

Analysis:

Did the TCP three-way handshake successfully complete?

Yes, the three way handshake was successfully completed. The packet capture showed a SYN packet was sent from the client , which was then followed by SYN/ACK packet in response from the server, and then a successful ACK packet from the client. This sequence demonstrates the TCP connection was successfully established before HTTP application-level communication began.

Figure 1: Successful TCP handshake

No.	Time	Source	Destination	Protocol	Length	Info
282	19.300213980	127.0.0.1	127.0.0.1	TCP	76	55084 → 8080 [SYN] Seq=0 Win=65495 Len=0
283	19.300238063	127.0.0.1	127.0.0.1	TCP	76	8080 → 55084 [SYN, ACK] Seq=0 Ack=1 Win=
284	19.300256688	127.0.0.1	127.0.0.1	TCP	68	55084 → 8080 [ACK] Seq=1 Ack=1 Win=65536

3.3 HTTP Traffic

What happened after the TCP handshake?

After the three-way handshake was completed, Firefox began communicating with the Python web server using HTTP. An HTTP GET request was sent to request the webpage from the server.

Did you observe an HTTP GET request? What did it request?

Yes, I observed an HTTP GET request from the browser. The request included GET / HTTP/1.1, which showed that Firefox was requesting the root page using HTTP/1.1

Was the application data exchanged?

Yes, the application data was exchanged after the TCP connection was established. Wireshark showed an HTTP GET request from the browser and an HTTP response. The response included HTTP/1.0 200 OK, which indicates that the server successfully returned the requested webpage.

What evidence in Wireshark supports your answer?

The packet capture showed that the TCP three-way handshake completed successfully before HTTP traffic appeared. Wireshark then displayed an HTTP GET request followed by an HTTP response containing 200 OK. This indicates evidence that the TCP connection was established, and packet communication occurred.

Figure 2: HTTP application traffic

285	19.300403856	127.0.0.1	127.0.0.1	HTTP	514 GET / HTTP/1.1
286	19.300412272	127.0.0.1	127.0.0.1	TCP	68 8080 → 55084 [ACK] Seq=1 Ack=447 Win=65536 Len=0 TSval=
287	19.301425067	127.0.0.1	127.0.0.1	TCP	224 8080 → 55084 [PSH, ACK] Seq=1 Ack=447 Win=65536 Len=156
288	19.301437067	127.0.0.1	127.0.0.1	TCP	68 55084 → 8080 [ACK] Seq=447 Ack=157 Win=65536 Len=0 TSva
289	19.301464192	127.0.0.1	127.0.0.1	HTTP	1155 HTTP/1.0 200 OK (text/html)

4. Investigation 2 — Failed TCP Connection

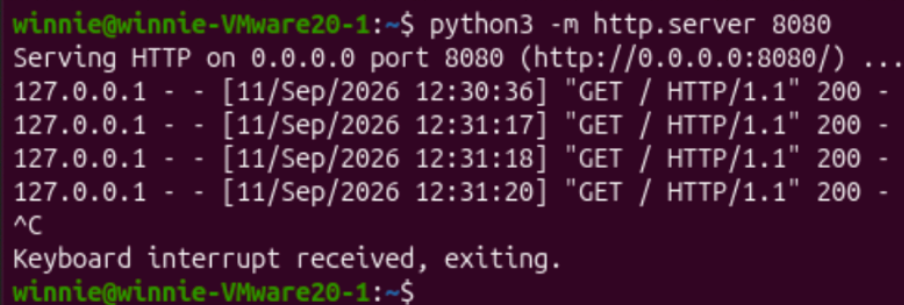
Objective:

The goal of this investigation was to determine what would happen when the Python web server was no longer running on port 8080. I wanted to compare the failed connection to the successful connection while using Wireshark to identify what the connection was no successful

What did you change before testing the connection again?

I stopped the Python HTTP server from the terminal, which caused the service that had been listening on port 8080 to stop accepting connections.

Figure 3: Python Server



```
winnie@winnie-VMware20-1:~$ python3 -m http.server 8080
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
127.0.0.1 - - [11/Sep/2026 12:30:36] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [11/Sep/2026 12:31:17] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [11/Sep/2026 12:31:18] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [11/Sep/2026 12:31:20] "GET / HTTP/1.1" 200 -
^C
Keyboard interrupt received, exiting.
winnie@winnie-VMware20-1:~$
```

What were you trying to determine from the failed connection?

I was trying to determine how the connection would act when there was no Python web server running on port 8080. I then used Wireshark to observe the packets generated during the failed attempt

4.1 Connection Attempt

What was the client's IP address?

127.0.0.1

What was the destination IP address?

127.0.0.1

What port was the client attempting to connect to?

8080

What was the client's source/ephemeral port?

35782

4.2 TCP Handshake

Did you observe a SYN packet?

Yes

Did you observe a SYN/ACK packet?

No

Did you observe an ACK completing the handshake?

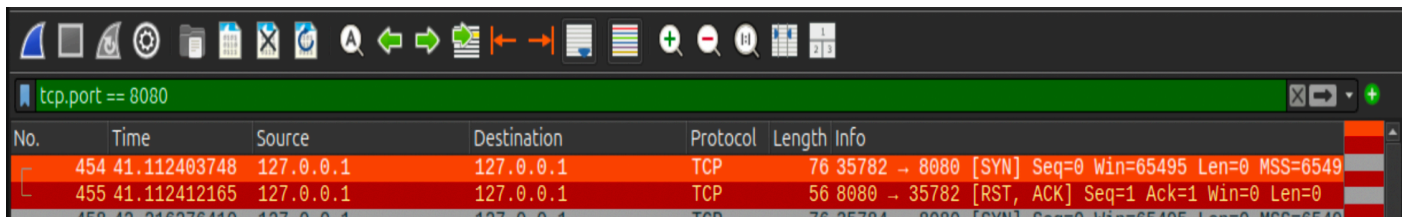
No

Packet	Source IP	Destination IP	Source Port	Destination Port	TCP Flags
SYN	127.0.0.1	127.0.0.1	35782	8080	SYN
Response	127.0.0.1	127.0.0.1	8080	35782	RST, ACK

Did the TCP three-way handshake complete?

No it didn't. When the client sent a SYN packet to 127.0.0.1 on port 8080, but in response, instead of receiving a SYN/ACK packet, the connection received a TCP RST/ACK. Because the SYN/ACK and final ACK that are required to complete a three-way handshake were not observed, the TCP connection was not established.

Figure 4: Failed Connection: SYN → RST/ACK



No.	Time	Source	Destination	Protocol	Length	Info
454	41.112403748	127.0.0.1	127.0.0.1	TCP	76	35782 → 8080 [SYN] Seq=0 Win=65495 Len=0 MSS=6549
455	41.112412165	127.0.0.1	127.0.0.1	TCP	56	8080 → 35782 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0

4.3 TCP Reset

Did you observe a TCP RST packet?

Yes

What did the RST indicate?

The RST indicated that the attempted TCP connection was reset immediately instead of being established. In this lab, the Python server had stopped before the connection was even attempted, showing that the connection could not be established.

What evidence supports this?

Wireshark showed a SYN connection attempt, which was followed by RST/ACK rather than the SYN/ACK that was shown in the previous successful connection

Figure 5: RST Filter

No.	Time	Source	Destination	Protocol	Length	Info
455	41.112412165	127.0.0.1	127.0.0.1	TCP	56	8080 → 35782 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
459	42.216282618	127.0.0.1	127.0.0.1	TCP	56	8080 → 35784 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
464	42.688327782	127.0.0.1	127.0.0.1	TCP	56	8080 → 35792 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
466	43.368111610	127.0.0.1	127.0.0.1	TCP	56	8080 → 35800 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
468	44.037713763	127.0.0.1	127.0.0.1	TCP	56	8080 → 35804 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
484	44.266066526	127.0.0.1	127.0.0.1	TCP	56	8080 → 35814 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
488	44.666649717	127.0.0.1	127.0.0.1	TCP	56	8080 → 35820 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
490	44.902061694	127.0.0.1	127.0.0.1	TCP	56	8080 → 35834 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
492	45.359062471	127.0.0.1	127.0.0.1	TCP	56	8080 → 35840 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
494	46.189035709	127.0.0.1	127.0.0.1	TCP	56	8080 → 43670 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
517	46.656516619	127.0.0.1	127.0.0.1	TCP	56	8080 → 43674 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
519	46.787388931	127.0.0.1	127.0.0.1	TCP	56	8080 → 43682 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
521	47.151994177	127.0.0.1	127.0.0.1	TCP	56	8080 → 43684 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
523	47.213365100	127.0.0.1	127.0.0.1	TCP	56	8080 → 43696 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
568	51.428896049	127.0.0.1	127.0.0.1	TCP	56	8080 → 43702 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
570	51.432091718	127.0.0.1	127.0.0.1	TCP	56	8080 → 43704 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
572	51.432149343	127.0.0.1	127.0.0.1	TCP	56	8080 → 43710 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
627	53.033483272	192.168.238.128	151.101.1.91	TCP	56	35574 → 443 [RST] Seq=104 Win=0 Len=0

4.4 Application Data

Was any HTTP application data exchanged during the failed connection?

No, the TCP connection was not successfully established, so the browser didn't reach the point of exchanging HTTP application data with the Python server.

What packet-level evidence supports your answer?

The packet capture showed the initial SYN connection attempt followed by a TCP RST/ACK.

There was no completed TCP three-way handshake and no HTTP GET request or HTTP communication following the failed connection attempt.

5. Successful vs. Failed Connection

	Successful	Failed
Destination IP	127.0.0.1	127.0.0.1

Destination Port	8080	8080
SYN Sent	Yes	Yes
SYN/ACK Received	Yes	No
ACK Received	Yes	No
HTTP Data Exchanged	Yes	No
RST Observed	No	Yes
Connection Established	Yes	No
Server Status	Running	Stopped

Comparison Analysis:

What were the biggest differences between the successful and failed connection?

In the successful connections, there was a clear TCP three way handshake present in the successful connection. The client sent a SYN, received a SYN/ACK, and responded with an ACK. Afterwards, HTTP application data was then exchanged.

In contrast, the failed connection did not have the same experience. The client sent a SYN but a RST/ACK was received instead of a SYN/ACK. The webpage was never truly able to connect to the server and because of this, the browser was unable to establish HTTP application data

Which packets appeared in the successful connection but not the failed connection?

The successful connection contained the complete TCP three-way handshake, including SYN, SYN/ACK, and ACK packets. It also contained HTTP application data, including GET request and HTTP response. The failed connection didn't contain the handshake or any HTTP data

How did these differences help you identify the problem?

The differences helped me determine that the issue occurred before the application could successfully communicate. The successful connection completed the TCP handshake and continued to HTTP communication while the failed connection did the opposite. Comparing the two captures makes it possible to identify where the connection stopped

6. Troubleshooting analysis

Based on your packet capture, what do you believe caused the connection to fail?

The connection failed because I ended the Python server that was listening on port 8080. After the server was stopped Firefox could no longer establish a TCP connection to port 8080.

Walk through how you diagnosed the problem using Wireshark

The first thing I analyzed was whether or not there was a three way handshake. I next used the `tcp.port == 8080`, to isolate traffic associated with the server port. In the successful connection, I identified the SYN, SYN/ACK and ACK packets and observed HTTP application data after the handshake.

For the failed connection, I exlaimed the traffic again using port 8080. I observed multiple SYN connection attempts but did not observe the SYN/ACK and ACK required to complete the handshake. Instead, RST/ACK packets were observed. By comparing the timeline of packets

from the successful and failed connections, I was able to determine that the failed connection occurred because service on port 8080 was no longer running

7. Wireshark Filters

Filter: `tcp.port == 8080`

Purpose: Used to display TCP traffic associated with port 8080 so that I could focus on the connection between Firefox and the Python HTTP server

What I observed:

During the successful connection I observed TCP three-way handshake along with HTTP traffic associated with port 8080. During the failed connection, I observed attempts and TCP reset packets

Filter: `tcp.flags.reset == 1`

Purpose: shows if there were any TCP packets containing the RST flag

What I observed:

The filter showed multiple TCP reset packets during the failed connection attempt. These packets were not present during the successful connection

Filter: `tcp.analysis.flags`

Purpose: Used to look for packets that Wireshark had identified with TCP analysis information or potential TCP-related conditions

What I observed:

No packets were displayed under this filter

Filter: tcp.analysis.retransmission

Purpose: Use to check whether any TCP retransmissions occurred during the connection attempts

What I observed:

No packets were displayed under this filter

8. Key Findings

What were the most important findings from the project?

Finding 1:

When the Python server was running on port 8080, Firefox was able to establish a TCP connection to 127.0.0.1:8080 and exchange HTTP data

Finding 2:

When the server was stopped, Firefox was no longer able to establish a TCP connection to port 8080

Finding 3:

The successful connection completed the TCP three-way handshake, while the failed connection did not

Finding 4:

The failed connection produced TCP RST/ACK packets instead of completing a normal handshake

Finding 5:

No HTTP application data was exchanged during the failed connection because the TCP connection was never successfully established

9. Key Takeaways

What did you learn from this project

This project helped me understand how Wireshark can be used to troubleshoot network connections at packet level. I learned how to identify a successful TCP three-way handshake by looking for the SYN, SYN/ACK, and ACK packets,

I also learned how a failed connection can look different in a packet capture. In this lab, the failed connection produced RST/ACK packets and did not complete the TCP handshake.

Because the connection was never established, there was no HTTP application data exchanged.

Comparing the successful and failed connections helped me understand how packet-level evidence can be used to determine where a connection is failing and identify potential causes of a network connectivity problem